

Technical Documentation: Advanced Web-Based Audio Analyzer

1. System Overview

The Audio Spectrum Analyzer is a professional-grade, zero-dependency client-side web application. It performs real-time Fast Fourier Transform (FFT) and time-domain analysis on live audio streams. Built natively with HTML5, Canvas 2D, and the Web Audio API, it provides zero-latency rendering of stereo audio streams captured directly from the user's microphone or internal system audio.

This document serves as the architectural reference, operational manual, and maintenance guide for future development.

2. Core Architecture & Signal Routing

2.1 Technologies Used

- **UI/Styling:** HTML5 & Tailwind CSS (via CDN).
- **Rendering Engine:** Native HTML5 `<canvas>` API driven by `requestAnimationFrame`.
- **Audio Engine:** HTML5 Web Audio API (`AudioContext`, `MediaStreamAudioSourceNode`, `AnalyserNode`, `ChannelSplitterNode`).

2.2 Signal Flow

1. **Input Generation:** `navigator.mediaDevices` captures raw streams via `getUserMedia` (Mic) or `getDisplayMedia` (System).
 2. **Gain Stage:** Audio is routed through a `GainNode` to allow pre-analysis digital amplification.
 3. **Channel Splitting:** The signal splits into three parallel analysis chains:
 - **Combined (L+R):** Web Audio natively downmixes the stereo stream to mono.
 - **Left Channel:** Isolated via `ChannelSplitterNode(2)` output 0.
 - **Right Channel:** Isolated via `ChannelSplitterNode(2)` output 1.
 4. **Analysis Nodes:** Each chain terminates in its own dedicated `AnalyserNode`, responsible for extracting either `FloatFrequencyData` (Spectrum) or `FloatTimeDomainData` (Oscilloscope).
-

3. Display Modes

The application features three toggleable canvases (Combined, Left, Right) that dynamically resize using `ResizeObserver`. The rendering engine supports four primary visualization modes:

1. **SPECTRUM (2D FFT)**: A standard frequency-domain line graph mapping Frequency (X) against Amplitude in dBFS (Y).
2. **WATERFALL (3D Spectrogram)**: A scrolling history mapping Frequency (X) against Time (Y). Amplitude is represented by color using a custom, high-performance "Jet/Turbo" Uint8ClampedArray colormap (Blue = quiet, Red = loud).
3. **BOTH**: The 2D spectrum line is drawn translucently on top of the scrolling 3D waterfall canvas.
4. **OSCILLOSCOPE (Time Domain)**: A raw waveform viewer mapping Time in milliseconds (X) against relative Voltage/Amplitude ± 1.0 (Y).

4. Technical Specifications & Capabilities

- **Frequency Range**: 0 Hz (DC) to the hardware Nyquist limit (typically ~22.05 kHz or 24 kHz).
- **Amplitude Range**: Visual span from -140 dBFS to 0 dBFS.
- **Resolution Bandwidth (FFT Size)**: Dynamically adjustable from 32 (2^5) to 32,768 (2^{15}). At maximum size, frequency resolution reaches ~1.34 Hz per bin.
- **Time Base (Scope)**: Adjustable window from 1ms up to 500ms.
- **Averaging**: Dual-layer. The internal API applies an 0.8 smoothing constant, combined with a user-adjustable Exponential Moving Average (EMA) ranging from 0% to 98%.

5. Detailed Feature Breakdown

5.1 Triggering System (Oscilloscope Mode)

To allow accurate phase comparison between Left and Right channels, the application features a phase-aligned trigger:

- **Trigger Source**: Users select the driving channel (Combined, L, or R).
- **Threshold Detection**: The rendering loop scans the time-domain buffer of the source channel for a rising edge that crosses the user-defined `scopeTriggerLevel`.
- **Phase Alignment**: Once the index of the crossing is found, *all active displays* begin drawing their waveforms starting from that exact microsecond index.

- **One-Shot:** Allows the user to arm the trigger. The engine silently monitors the stream; upon threshold crossing, it captures the exact frame, disarms itself, and forces a global `isFrozen` state for measurement.

5.2 Measurement Tools & Smart Cursors

The X and Y cursors dynamically re-map their mathematical contexts based on the active mode:

- **Spectrum Context:**
 - * X-Cursors calculate absolute Hz and ΔF .
 - Y-Cursors calculate absolute dB and ΔA .
 - **Peak Detection:** Algorithm finds the top 3 highest local maxima, enforcing a minimum spatial separation (50Hz or 2% screen width) to prevent peak-clumping.
- **Oscilloscope Context:**
 - X-Cursors calculate Time (ms) and ΔT , and automatically extrapolate the Frequency ($1 / \Delta T$) in Hz.
 - Y-Cursors calculate relative Voltage (ΔV) factoring in the active `GainNode` multiplier to display the *true original input voltage*.

5.3 Memory & Storage System

- **Capture to Memory:** Takes a deep copy of the current state, including `Float32Arrays` for time/frequency data. Because Waterfalls represent historical data over time rather than a single array, the system generates base64 PNG images (`cvs.toDataURL`) of the active waterfall canvases.
- **Overlay:** Renders the captured memory data in a distinct red color *behind* the live blue trace, allowing for real-time A/B phase and EQ comparisons.
- **File I/O:** Serializes the `memorySnapshot` object into JSON format, creates a virtual anchor link, and forces a file download. The Load function parses a user-provided JSON file and executes a **RESTORE & FREEZE** command to view historical data.

6. Maintenance & Troubleshooting Guide

If modifying the code in the future, adhere to these architectural quirks:

1. **The Chromium "Silent Tab" Bug:**
 - * *Issue:* Browsers (like Chrome) will heavily throttle or completely silence a `MediaStreamTrack` from `getDisplayMedia` if the audio isn't explicitly consumed by an HTML media element.
 - *Fix:* The code maintains a `dummyAudioEl` (a hidden, muted `<audio>` tag). Do not remove this, or system audio capture will flatline.

2. Infinite Loop Protection (Performance):

- *Issue:* At an FFT size of 32,768, looping through the data array 60 times a second can cause frame-drops.
- *Fix:* The `drawTrace` function uses a math optimization (`startB` and `endB`) calculated from the `zoomMinFreq` and `zoomMaxFreq`. The `for` loop skips processing data that falls outside the user's visible canvas area. Ensure this logic is maintained if altering the drawing algorithms.

3. Scope Data Exhaustion:

- *Issue:* If the user sets the Time Base to 500ms, the required sample count may exceed the `fftSize` buffer length provided by the Web Audio API.
- *Fix:* `drawOscilloscope` calculates `samplesToDraw` dynamically based on the sample rate and buffer length, capping the draw loop to prevent undefined array indexing.

7. Roadmap for Future Improvements

When expanding the tool, consider the following upgrades:

1. **Windowing Function Controls:** The Web Audio API permanently applies a Blackman window to its FFT data. To offer Hanning, Hamming, or Flat windowing, you would need to bypass `getFloatFrequencyData`, capture raw time-domain data, and write a custom JavaScript/WebAssembly FFT algorithm.
2. **Data Export to CSV:** Add a button in the Memory panel that iterates through the active `displayDataArray`, pairs it with the corresponding frequency bins, and exports a raw `.csv` string for Python/Excel analysis.
3. **Lissajous (X-Y) Phase Plot:** Add a 5th display mode that uses the Left channel to drive the X-axis and the Right channel to drive the Y-axis. This is the industry standard visualizer for stereo phase correlation and stereo-width checking.
4. **Web Workers:** If implementing custom FFTs or massive historical waterfall buffers, offload array processing to a Web Worker to ensure the main UI thread maintains 60 FPS.